

BoxAdmin — Fase 5: Pagos y comunicación

Especificación técnica para agente de desarrollo (Claude Code)

Septiembre 2026

Contexto

Con el ciclo operativo, la recurrencia, el self-service del alumno y el profesor real ya funcionando, esta fase cierra el **ciclo de cobro**: estados de pago, aprobación de comprobantes (ya subidos desde Fase 3), señas, y comunicación por email/push (confirmaciones, recordatorios, vencimientos).

Objetivo de esta fase: el admin tiene visibilidad y control completo del estado de pago de cada alumno, y el sistema envía automáticamente los emails/notificaciones clave del ciclo de vida de una reserva y un pack, sin intervención manual repetida.

Criterio de éxito: un alumno con pack por vencer recibe un email de recordatorio automático; al aprobar su comprobante, su estado pasa a “al día” y se desbloquean sus próximas clases; el admin puede configurar su propio SMTP sin exponer credenciales en ningún log.

1. Dependencias adicionales

```
pnpm add nodemailer handlebars
pnpm add -D @types/nodemailer
pnpm add web-push # para notificaciones push del PWA
```

web-push requiere generar un par de claves VAPID una sola vez:

```
pnpm dlx web-push generate-vapid-keys
```

Guardar VAPID_PUBLIC_KEY y VAPID_PRIVATE_KEY en .env (nunca versionadas).

2. Modelo de datos de esta fase (Prisma)

```
model ConfiguracionSMTP {
  tenantId      String  @id
  tenant        Tenant  @relation(fields: [tenantId], references: [id])

  emailOrigen   String
  claveSMTPCifrada String // cifrado con una clave de aplicación (AES-256-GCM), nunca en
  ↪ texto plano
  emailDestino  String? // para copiar notificaciones internas del salón

  updatedAt     DateTime @updatedAt

  @@map("configuraciones_smtp")
}
```

```

}

model PlantillaEmail {
  id          String    @id @default(cuid())
  tenantId    String

  tipo        String    // "confirmacion" | "cancelacion" | "recordatorio_pago" |
  ↪ "lista_espera" | "vencimiento_pack" | "ultima_clase"
  asunto      String
  cuerpoHtml  String

  @@unique([tenantId, tipo])
  @@map("plantillas_email")
}

model Pago {
  id          String    @id @default(cuid())
  tenantId    String
  perfilId    String
  perfil      Perfil    @relation(fields: [perfilId], references: [id])

  monto       Decimal   @db.Decimal(10, 2)
  metodo      String    // "efectivo" | "transferencia" | "otro"
  esSena      Boolean   @default(false)
  comprobanteId String?
  comprobante Comprobante? @relation(fields: [comprobanteId], references: [id])

  registradoPor String
  createdAt   DateTime @default(now())

  @@index([tenantId, perfilId])
  @@map("pagos")
}

model SuscripcionPush {
  id          String    @id @default(cuid())
  perfilId    String
  perfil      Perfil    @relation(fields: [perfilId], references: [id])
  endpoint    String
  p256dh      String
  auth        String
  createdAt   DateTime @default(now())

  @@unique([perfilId, endpoint])
  @@map("suscripciones_push")
}

```

Migración:

```
pnpm dlx prisma migrate dev --name fase5_pagos_comunicacion
```

Seguridad de credenciales SMTP: cifrar claveSMTPCifrada con una clave de aplicación (APP_ENCRYPTION_KEY en .env, nunca en el repo) usando `crypto.createCipheriv('aes-256-gcm', ...)`. Nunca loguear el valor descifrado, ni siquiera en logs de debug — el manual de TurnoFit ya advertía sobre esto explícitamente y es una buena práctica a mantener.

3. Endpoints de esta fase

3.1 Pagos

Método	Ruta
POST	/pagos
GET	/pagos?perfilId=&desde=&hasta=
PATCH	/usuarios/:id/estado-pago

3.2 Comprobantes (extiende Fase 3)

Método	Ruta
PATCH	/comprobantes/:id/aprobar

3.3 Configuración de email

Método	Ruta
PUT	/config/smtp
GET	/config/plantillas
PUT	/config/plantillas/:tipo

3.4 Notificaciones push

Método	Ruta
POST	/push/suscripcion
DELETE	/push/suscripcion

4. Jobs de comunicación (BullMQ)

src/jobs/

- └ recordatorio-pago.processor.ts # corre diariamente, busca perfiles con pagoAlDia=false y
- └ vencimiento-pack.processor.ts # corre diariamente, busca packs con vigenciaHasta próxima
- └ notificacion-reserva.processor.ts # dispara al crear/cancelar una reserva (confirmación/ca
- └ notificacion-lista-espera.processor.ts # dispara cuando se libera un cupo y hay alguien en e

Cada processor: 1. Arma el contexto de datos (nombre del alumno, turno, fecha, etc.) 2. Resuelve la PlantillaEmail del tenant para el tipo correspondiente (o usa una plantilla default si el tenant no configuró una propia) 3. Renderiza con Handlebars 4. Envía por nodemailer usando la ConfiguraciónSMTP del tenant (descifrando la clave solo en memoria, en el momento del envío) 5. Si el perfil tiene SuscripcionPush activa, envía también la notificación push equivalente vía web-push

Programar los jobs recurrentes con repetición de BullMQ:

```
await recordatorioPagoQueue.add(  
  'diario',  
  {},  
  { repeat: { pattern: '0 9 * * *' } }, // todos los días a las 9am  
);
```

5. Checklist de aceptación de la Fase 5

- ☐ El admin puede configurar su SMTP propio y el sistema valida la conexión antes de guardar.
 - ☐ Ninguna credencial SMTP aparece en texto plano en logs, respuestas de API o base de datos (verificar cifrado).
 - ☐ Aprobar un comprobante crea el Pago correspondiente y actualiza el estado del alumno automáticamente.
 - ☐ Se puede registrar un pago manual (efectivo/transferencia) sin comprobante asociado.
 - ☐ El job de recordatorio de pago corre diariamente y envía email solo a los perfiles con pago pendiente real.
 - ☐ El job de vencimiento de pack notifica con la anticipación configurada.
 - ☐ Confirmación y cancelación de reserva disparan el email correspondiente usando la plantilla del tenant (o la default).
 - ☐ Las notificaciones push llegan correctamente a un dispositivo suscripto vía PWA.
 - ☐ Tests: cifrado/descifrado de credenciales SMTP, renderizado de plantilla con datos reales, aprobación de comprobante → pago → estado, job de vencimiento con distintos escenarios de fecha.
-

6. Qué NO entra en esta fase

Estadísticas y reportes agregados (caja del mes, ocupación, liquidación en pesos), web pública del salón, check-in QR y tienda — llegan en la Fase 6.